

E-Panchayat

An AI-assisted decision support system
for Gram Panchayat administration

How the system works, how it was built, and why each decision was made the way it was, followed by the questions a senior software engineer is likely to ask about it, with worked answers, algorithms, complexity analysis and code.

Sem-7 Capstone · Group BCC28

MIT School of Computing, MIT-ADT University, Pune

Guide: Prof. Jyoti Gavhane

Scope: 23 villages of Haveli block, Pune district, Maharashtra

Repository state documented: commit 897818c · October 2026

Contents

Reading this handbook	4
1. The problem and the scope	5
What a Gram Panchayat actually has to do	5
What E-Panchayat does	5
Numbers worth remembering	5
2. How it was built: the project's evolution	6
Phase 1: a frontend prototype (August 2026)	6
Phase 2: a FastAPI backend becomes the source of truth	6
Phase 3: hardening, driven by actually running it	6
3. Architecture and technology choices	8
The three tiers	8
Why each technology was chosen	8
Backend layout	9
4. The data model	10
Decisions worth defending	10
5. Authentication and sessions	11
Tokens	11
Revoking a stateless token	11
Sign-in throttling	11
Password reset over the counter	12
6. Authorisation: roles and village isolation	13
Two layers, and why both are needed	13
Failing closed	13
7. The eligibility engine	14
Rules are data, not code	14
Three-valued logic	14
How any_of works	14
Document matching	15
Why no LLM in this path	15
8. The retrieval-augmented assistant	16
Indexing: records become linked sentences	16
Search: scope first, then rank	16

Graph expansion	16
The privacy boundary: the model may see the village, never the villager	17
Graceful degradation	17
Resilient model calls	17
9. Grievance classification	18
10. Document readers and resident sign-up	19
Gram Sabha transcript reader	19
Government Resolution to scheme	19
Resident sign-up and record matching	19
11. The audit trail	20
12. The frontend	21
A typed client with one transparent refresh	21
Two hooks instead of a state library	21
Maps, charts and language	21
13. Testing, deployment and operations	22
Testing	22
Deployment	22
14. Honest limitations and how it would scale	23
What the system does not have	23
From one block to a whole state	23
15. Interview question bank: data structures and algorithms	24
16. Interview question bank: system design and security	28
17. Coding exercises drawn from the project	30
Exercise 1: evaluate nested eligibility criteria	30
Exercise 2: a sliding-log rate limiter	30
Exercise 3: single-flight token refresh	31
Exercise 4: top-k with graph expansion	31
Exercise 5: merge overlapping project timelines	31
18. Behavioural and viva questions	33
Quick-fire viva answers	33
19. Glossary	35

Reading this handbook

This handbook is written for two readers. The first is anyone who has to understand E-Panchayat well enough to present it, defend it in a viva, or extend it next semester. The second is the interviewer sitting across the table, the kind of senior engineer at a company like Google who will not be satisfied with a feature list and will want to know *why* the system is shaped the way it is, what it costs in time and space, and where it breaks.

Every claim here was checked against the code in the repository at commit 897818c. Where the project made a trade-off, the trade-off is stated alongside the decision. That habit runs through the codebase itself, and it is the single most useful thing to carry into an interview: a system you can criticise honestly is a system you clearly understand.

Part	What it covers	Use it for
1 to 3	The problem, the scope, how the project evolved, and the architecture	The opening five minutes of any presentation
4 to 11	Each subsystem in depth: data model, security, eligibility, retrieval, classifier, document readers, audit, frontend	Viva questions on a specific module
12 to 13	Testing, deployment and operations	"How do you know it works?" and "How is it run?"
14	Honest limitations and how the design would scale	System design follow-ups
15 to 18	Interview question bank: DSA, system design, coding exercises with solutions, behavioural	Interview preparation
19	Glossary	Quick reference

1. The problem and the scope

What a Gram Panchayat actually has to do

A Gram Panchayat is the elected local government of a village, the lowest tier of India's three-level Panchayati Raj system. In Maharashtra its office keeps the village register, receives complaints about water, drains, roads and streetlights, runs development works against a budget, holds the Gram Sabha (the assembly of all adult residents) and records its decisions, and helps residents claim central and state welfare schemes.

In practice most of that lives in paper registers and spreadsheets. The questions an officer is asked every day are simple to state and slow to answer: *Is this widow eligible for the Sanjay Gandhi Niradhar pension? Which ward has had the most water complaints this month? How much of the road budget has been spent? What did the last Gram Sabha decide about the drainage work?*

What E-Panchayat does

- **Multi-village, permission-scoped records.** A real state, district, block and village hierarchy with official LGD (Local Government Directory) codes for 23 villages of Haveli block, Pune. An officer sees one Gram Panchayat, an admin sees the block, and a resident sees only their own file.
- **Welfare eligibility as data.** 29 real central and Maharashtra schemes, each with machine-readable rules. A deterministic engine returns one of four verdicts and explains which rule produced it.
- **A retrieval-augmented assistant.** Questions in English or Marathi are answered from the Panchayat's own records, scoped to the asker, with sources attached.
- **Grievance management** with automatic categorisation, priority and department routing, plus a full status history.
- **Document readers** that turn Gram Sabha minutes into decisions and action items, and a Government Resolution into a proposed scheme that waits for officer approval.
- **Resident sign-up with verification**, over-the-counter password reset, an audit trail, analytics dashboards and a GIS map.

All resident data is synthetic. The villages, LGD codes and schemes are real and cited. The ten residents, their incomes, families and documents are invented. This matters in an interview: it shows the team understood that real welfare data should never be used to demo a student project.

Numbers worth remembering

Item	Count	Item	Count
Villages (with LGD codes)	23	Seeded welfare schemes	29
Villages with verified map centre	18	Seeded residents (synthetic)	10
Backend tests (all passing)	284	Alembic migrations	10
Embedded knowledge chunks	about 67	Embedding width	768 floats
Backend source (Python)	about 8,200 lines	Frontend source (TSX/TS)	about 15,000 lines

2. How it was built: the project's evolution

The git history tells a clear story, and it is a better story than "we designed it and built it". The project started as a frontend prototype, ran into the limits of that design, and was rebuilt around a proper backend. Being able to narrate this is valuable, because interviewers care far more about how a team reacts to its own mistakes than about a clean first draft.

Phase 1: a frontend prototype (August 2026)

The first version was a React single-page app styled as a government portal. Data lived in the browser's `localStorage` and was synchronised to Supabase on a best-effort basis. The AI features called Gemini straight from the browser.

When the team looked at it critically, three serious problems surfaced:

- **Writes were failing silently.** Four of the five Supabase tables rejected every write because the SQL schema and the TypeScript models described different data. The errors were swallowed into `console.warn`, so a broken save looked exactly like a successful one.
- **There was no real authentication.** The officer password was compared in client-side JavaScript, and typing any citizen ID at the login screen opened that resident's file.
- **The API key was in the bundle.** Vite compiles any `VITE_` variable into the JavaScript, so the only configuration in which the AI worked was one where anyone could read the key.

Phase 2: a FastAPI backend becomes the source of truth

The rebuild moved every rule to the server. FastAPI, SQLAlchemy 2.0 and Alembic replaced the client-side store; JWT authentication and role guards replaced the password check in JavaScript; the Gemini key moved behind the API. The frontend was migrated screen by screen onto a typed client (`src/lib/api.ts`) whose failures throw rather than warn.

Phase 3: hardening, driven by actually running it

The later commits are where the engineering maturity shows. Most of them fix something that only became visible by using the system as a real officer or resident would:

Commit theme	What was found	What changed
Semantic retrieval, privacy	Indexing residents would export the village register to a third party on every index run	Residents excluded from the index; personal facts flagged and never sent to the model
Meter /auth/login	Login was an unlimited password oracle	Database-backed throttling, five failures per email, twenty per IP, per fifteen minutes
Password reset	No email or SMS gateway exists	Officer-issued one-time codes at the counter, sessions revoked on reset
Audit trail	No record of who opened whose file	Middleware that records every write and every single-record read
Stop the assistant filling the audit trail	Each question wrote two noise rows	Read-only POSTs exempted by an explicit allow-list
Model health check	A retired model name returned 404 for days while /health said all was well	A separate endpoint that makes one real call

Commit theme	What was found	What changed
Close village isolation	Lists were scoped, but records could still be opened by ID	A by-ID guard on every route, proven by 24 cross-village tests

The interview line: "Our tests all passed while the bug existed, because every seeded resident lived in one village, so 'the neighbouring officer sees an empty list' looked true everywhere. We learned that tests only prove what your fixtures can express, and we added fixtures specifically to break that assumption."

3. Architecture and technology choices

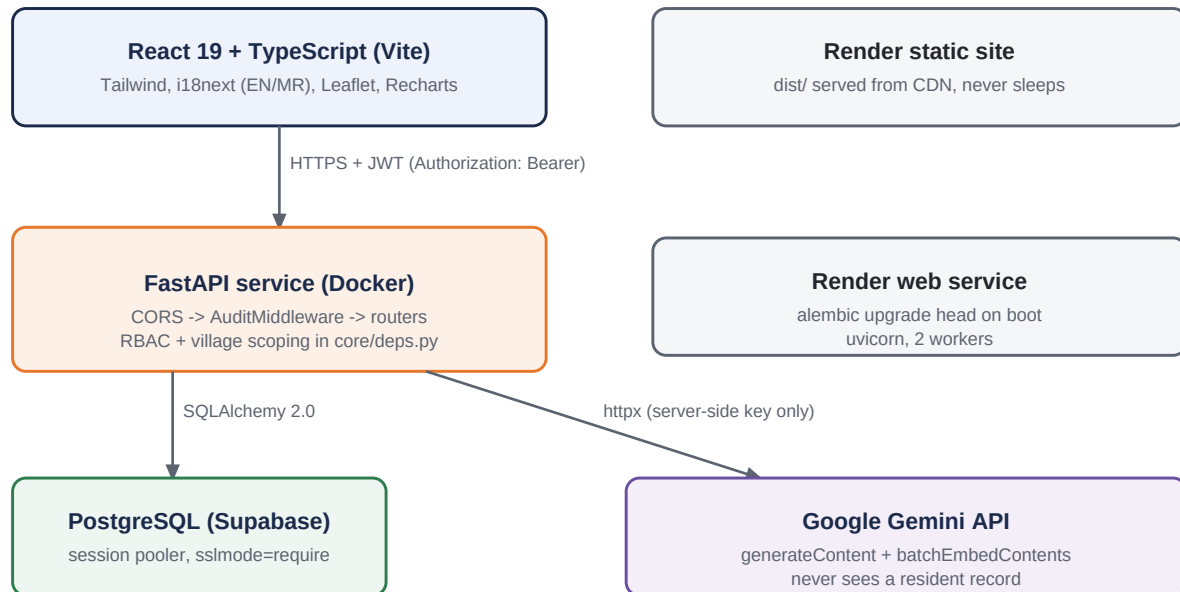


Figure 1. Deployment and request path.

The three tiers

A static React bundle talks to a stateless FastAPI service over HTTPS, carrying a JWT in the Authorization header. The API owns every rule and every query and is the only component that talks to PostgreSQL or to Gemini. Because the API is stateless (sessions are tokens, throttle counters are rows), it can run as several workers or several instances without coordination.

Why each technology was chosen

Choice	Why it was chosen	What was given up / alternatives considered
FastAPI (Python 3.11)	Dependency injection makes auth and scoping composable per route; Pydantic validation at the edge; free OpenAPI docs at /docs for demos; async for the LLM calls; Python is where the PDF/DOCX parsing libraries are	Django is heavier and its ORM is less explicit; Node/Express would split the team's language and lacks equivalent document tooling
SQLAlchemy 2.0 + Alembic	Typed Mapped[] models are the single schema source of truth; migrations are versioned and run automatically on boot	The prototype's drift between a hand-written SQL file and TypeScript types is exactly what this prevents
PostgreSQL on Supabase	Relational data with real foreign keys, free hosting, already in use. JSON columns hold scheme criteria and embeddings	SQLite is used in tests for speed; MongoDB would lose joins and constraints the hierarchy depends on
React 19 + TypeScript + Vite	Typed API client catches contract drift at build time; Vite gives fast builds and fingerprinted assets	Next.js would add server rendering the app does not need
No Redux, no React Query	Two small hooks (useQuery, useMutation) cover fetch-on-mount, loading, error and refetch	React Query adds caching and deduplication; worth adopting if screens start sharing data heavily
Tailwind 4	Consistent government-portal styling without a CSS architecture to maintain	Utility classes make markup longer

Choice	Why it was chosen	What was given up / alternatives considered
i18next	English and Marathi; Marathi is what most residents read	Only two languages are supported today
Leaflet + OpenStreetMap	No API key, no billing account, no watermark	Google Maps would need a key and a card
Gemini 2.5/3.x Flash	Free tier, structured JSON output via response schema, embeddings from the same provider	Free tier may use submitted content, which is why personal data never reaches it
JWT (HS256) + bcrypt	Stateless sessions suit a sleeping free-tier server; bcrypt is deliberately slow	Stateless tokens cannot be revoked, solved with a per-user tokens_valid_from timestamp
Render + Docker	Whole deployment declared in render.yaml; reviewable and rebuildable	Free API instance sleeps after 15 minutes idle

Backend layout

Where things live

```

backend/app/
  api/routes/    one module per resource: auth, citizens, schemes, grievances,
                 projects, sabha, documents, villages, analytics, assistant, audit
  core/          config.py (settings), security.py (JWT, bcrypt, reset codes),
                 deps.py (current user, role guards, village scoping)
  services/
    eligibility.py the rule engine (no model involved)
    retrieval.py   question -> scoped facts, keyword and semantic paths
    graph.py       cosine similarity and graph expansion
    indexer.py     rows -> linked, embeddable chunks (no residents)
    scheme_reader.py Government Resolution -> pending scheme
    transcript.py  Gram Sabha minutes -> decisions and action items
    classifier.py  grievance -> category, priority, department
    ratelimit.py  sign-in throttling, counted in the database
    audit.py       the audit trail middleware
    llm.py         the only module that talks to Gemini
  models.py       SQLAlchemy models, the single schema source of truth
  schemas.py      Pydantic request/response models (camelCase on the wire)

```

One design rule runs through this layout: **each external dependency has exactly one door**. Only `llm.py` talks to Gemini, only `deps.py` decides who can see what, and only `graph.semantic_search()` knows how vectors are stored. That is what makes later changes (a new model, pgvector, a new role) a one-file job.

4. The data model

The schema is defined in `backend/app/models.py` and evolved through ten Alembic migrations. Primary keys are readable strings such as `cit_102` or `griev_201`, which make demos and logs easy to follow. (As Section 6 explains, guessable IDs are exactly why every by-ID route needs its own permission check.)

Group	Tables	Notes
Administrative hierarchy	states, districts, blocks, villages	Real LGD codes; Census 2011 population; optional lat/long
People	families, citizens, users, registration_requests	A user optionally links to one citizen; officers carry a <code>village_id</code>
Welfare	schemes, citizen_documents	<code>Scheme.criteria</code> is JSON; <code>required_documents</code> is a JSON list
Operations	grievances, grievance_events, projects, facilities	<code>grievance_events</code> is an append-only status history
Gram Sabha	sabha_meetings, sabha_action_items	Decisions and assignable action items
Retrieval	knowledge_chunks	Text, links (JSON), embedding (JSON), <code>village_id</code> , <code>indexed_at</code>
Security	auth_attempts, password_resets, audit_events	Throttling counters, hashed one-time codes, the audit trail

Decisions worth defending

- **Criteria as JSON, not columns.** Scheme rules vary wildly in shape. A JSON dictionary lets a new scheme be an insert rather than a migration, and the rule vocabulary is validated in code (Section 7).
- **Embeddings as JSON, not pgvector.** At about 67 chunks a linear scan takes well under a millisecond. `pgvector` would add an extension dependency to every environment for no visible gain. The note on `KnowledgeChunk` says exactly when this stops being true (around 100,000 chunks).
- **JSON(`none_as_null=True`) on the embedding column.** By default SQLAlchemy stores Python `None` as the JSON value `null`, which is not SQL `NULL`, so `embedding IS NOT NULL` would match every unindexed row. A test caught this.
- **A unique constraint on (entity_type, entity_id)** plus a deterministic chunk ID (`kc_` + first 16 hex characters of SHA-1 of the key) makes re-indexing an idempotent upsert.
- **Timezone-aware timestamps everywhere**, with `as_utc()` normalising SQLite's naive values. Without it, a server set to IST would let a revoked token live five and a half hours longer.

5. Authentication and sessions

Tokens

Sign-in returns an access token (60 minutes) and a refresh token (7 days), both HS256 JWTs carrying `sub`, `type`, `iat` and `exp`. The access token also carries `role` and `citizen ID`. `decode_token()` rejects a token of the wrong type, so a refresh token cannot be used as an access token.

Revoking a stateless token

The classic weakness of JWTs is that nothing on the server can end a session. If a password is reset because someone else had the account, that someone would stay signed in for up to a week. E-Panchayat solves this with one column: `users.tokens_valid_from`. Every authenticated request already loads the user row, so the check costs nothing extra:

backend/app/core/deps.py

```
def token_is_revoked(user: User, payload: dict) -> bool:
    valid_from = as_utc(user.tokens_valid_from)
    if valid_from is None:
        return False
    issued_at = payload.get("iat")
    if issued_at is None:
        return True # cannot be placed in time, so cannot post-date revocation
    return issued_at < valid_from.timestamp()
```

`iat` has one-second resolution, so a token minted in the same second as the revocation survives. That is deliberate: the alternative would reject the fresh token issued by the sign-in immediately after a reset.

Sign-in throttling

Without a limit, `/auth/login` is a password oracle: slow per guess because of `bcrypt`, but free to repeat. The limiter counts *failures* in a fixed lookback window and refuses with HTTP 429 before `bcrypt` is ever run.

Rule	Value	Reason
Failures per email per window	5	Stops focused guessing against one account
Failures per IP per window	20	Stops spraying many accounts from one source
Window	15 minutes	Short enough that a deliberate lockout is bearable
Counted outcomes	<code>bad_password</code> , <code>no_account</code>	A refusal never extends the lockout, so it always expires
Storage	<code>auth_attempts</code> table	An in-memory counter would reset on every cold start of the sleeping server

- **Unknown addresses are throttled exactly like real ones.** Otherwise the throttle itself would reveal which emails are registered.
- **IP is read from the right-most X-Forwarded-For hop**, the one Render's own proxy appended. The left-most entry is whatever the client chose to send.
- **Registration responses are identical** whether or not the email already exists.

Password reset over the counter

There is no email or SMS gateway, and a reset flow whose message never arrives is worse than none. So the flow uses the channel a Panchayat actually has: the office counter. An officer verifies the resident against the register and issues a one-time code, shown once. The resident redeems it for a password of their own choosing, so the officer never learns the password.

backend/app/core/security.py

```
_CODE_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789" # 0 I L S 0 1 removed
_CODE_LENGTH = 10 # 30^10 ~ 5.9e14 ~ 49 bits

def generate_reset_code() -> str:
    raw = "".join(secrets.choice(_CODE_ALPHABET) for _ in range(_CODE_LENGTH))
    return f"{raw[:5]}-{raw[5:]}" # 'K7MPQ-4XRTV'
```

- Uses secrets, never random, because Mersenne Twister output is reconstructable.
- Stored only as a bcrypt hash, expires in 24 hours, single use, and voided when a new one is issued.
- An officer may reset only residents of their own village; only an admin may reset staff. Otherwise one village login would become a route to the whole block.
- Redeeming is metered separately, so failing a code does not lock out a password the user still remembers.

6. Authorisation: roles and village isolation

All access control lives in `core/deps.py` and is applied as FastAPI dependencies. The frontend hides buttons; the server is what refuses. A route that does not depend on a guard is public, so the list of guards is the list to audit.

Role	Scope	How it is enforced
admin	Every village in the block, plus the district rollup	<code>village_scope()</code> returns <code>None</code> , meaning no filter
officer	One Gram Panchayat	<code>village_scope()</code> returns the officer's <code>village_id</code>
citizen	Their own record, documents, grievances and eligibility	<code>assert_can_read_citizen()</code> compares citizen IDs
anonymous	Login, register, refresh, health only	Every other route depends on <code>get_current_user</code>

Two layers, and why both are needed

Layer one: scoped lists. Every list endpoint adds `WHERE village_id = :scope`.

Layer two: by-ID guards. A list that leaves a record out protects nothing if the record can still be opened by ID, and the seeded IDs are guessable. Every route that opens, edits or deletes one record calls `assert_can_access_village()` with that record's village. This is the classic **IDOR** (Insecure Direct Object Reference) defence, number one on the OWASP Top 10 as part of Broken Access Control.

Failing closed

An earlier version returned `user.village_id` for everyone, and callers treated `None` as "no filter". An officer with no village assigned therefore saw the *entire block*. Now `None` is the admin's answer and nobody else's; any other account without a village gets a 403. A missing value should shrink access, never widen it.

One subtle detail: when an officer asks for a resident who does not exist, the guard returns quietly and lets the route raise 404. Refusing with 403 instead would tell the officer which IDs exist in other villages.

Test file	Tests	What it proves
<code>test_village_isolation.py</code>	24	The neighbouring officer can neither read nor change another village's records, route by route
<code>test_api.py</code>	87	End-to-end behaviour of every resource, including role refusals
<code>test_password_reset.py</code>	28	Who may reset whom, expiry, single use, session revocation
<code>test_rate_limit.py</code>	13	Thresholds, non-renewal of lockouts, unknown-email parity

7. The eligibility engine

Precedence is fixed in Assessment.status: Ineligible > Needs Review > Missing Documents > Eligible

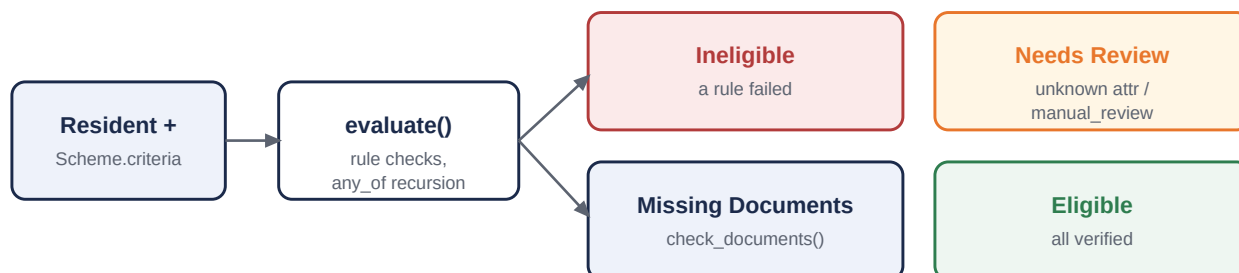


Figure 2. One resident, one scheme, one verdict.

Rules are data, not code

The prototype hard-coded rules per scheme in a chain of `if` statements, so an unknown scheme silently fell through to a bare income test. Now each scheme carries a criteria dictionary:

An example criteria dictionary

```
# Sanjay Gandhi Niradhar: income under Rs 21,000 OR on the BPL list
{
  "min_age": 18,
  "any_of": [ {"max_income": 21000}, {"requires_bpl": true} ]
}
```

The supported vocabulary (age band, income band, gender, marital status, social category, BPL, SECC-2011 listing, ration card type, land holding, disability percentage, occupation include/exclude, ward, household head, `any_of` and `manual_review`) was widened after reading real scheme notifications, because almost no real scheme decides on age and income alone.

Three-valued logic

The most important design idea is that a rule can **pass**, **fail** or be **unknown**. If a scheme needs a ration card type and the record does not hold one, the engine does not guess. It reports the attribute as unknown and the verdict becomes *Needs Review*. Silently passing would promise a benefit; silently failing would deny one.

How `any_of` works

`any_of` is evaluated recursively, so groups can nest. The combination logic is a small three-valued OR:

backend/app/services/eligibility.py

```
if branches := criteria.get("any_of"):
    results = [evaluate(citizen, branch) for branch in branches]
    if any(r.passed and not r.unknown for r in results):
        pass # a branch is cleanly satisfied
    elif any(r.unknown for r in results):
        out.unknown.extend(sorted({a for r in results for a in r.unknown}))
    else:
        fail("None of the alternative conditions are met (...)", ...)
```

Complexity. For one resident and one scheme, evaluation is $O(R)$ in the number of rule nodes R in the criteria tree. Screening a village is $O(C \times S \times R)$ for C residents and S schemes, plus document matching. At 10 residents and 29 schemes that is trivial; at district scale (say 50,000 residents) it is still linear and

embarrassingly parallel.

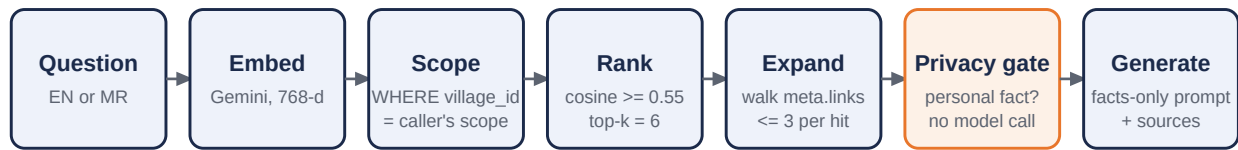
Document matching

Once criteria pass, required documents are matched against the resident's uploads by normalised name (lowercase, alphanumerics only) with bidirectional substring checks, preferring a *Verified* copy if two exist. This replaced a first-eight-characters trick that matched "Land ownership 7/12" against anything beginning "Land own".

Why no LLM in this path

A welfare decision has to be reproducible and auditable. The same inputs must give the same verdict tomorrow, and an officer must be able to point to the rule that produced it. A generated sentence is neither. `explain()` therefore builds the explanation, in English or Marathi, from the very facts the engine used.

8. The retrieval-augmented assistant



Fallbacks: no index, no key, embedding failure or zero hits above the floor -> keyword routing (gather) over the same scoped queries.

Citizen questions: keyword pass runs first so the rule engine's verdicts are included; semantic hits are layered on top.

Model unavailable: `plain_answer()` renders the same retrieved facts as text. It degrades; it does not invent.

Figure 3. The semantic retrieval pipeline and its fallbacks.

Indexing: records become linked sentences

`services/indexer.py` renders villages, schemes, projects, facilities, grievances and Gram Sabha meetings as natural-language sentences, the way a person would describe them, because that is what a question has to match. Each chunk also carries `meta.links`: typed edges to related records ("in village", "filed by"). These edges turn a flat vector store into a small knowledge graph.

- **Batching:** 32 texts per `batchEmbedContents` call, with a per-text fallback if the model has no batch endpoint.
- **Fixed width:** 768 dimensions requested explicitly, so a provider default change cannot silently make old and new vectors incomparable.
- **Incremental:** a chunk whose source row has not changed since `indexed_at` is skipped.

Search: scope first, then rank

`backend/app/services/graph.py` (simplified)

```

def semantic_search(db, query_vector, user, village_id, limit=6, expand=True):
    visible = _visible_chunks(db, user, village_id)  # permission WHERE clause
    scored = [Hit(c, cosine(query_vector, c.embedding or [])) for c in visible]
    scored = [h for h in scored if h.similarity >= MIN_SIMILARITY]  # 0.55
    scored.sort(key=lambda h: h.similarity, reverse=True)
    top = scored[:limit]
    return top + _expand(db, top, visible) if (top and expand) else top
  
```

Scoping happens *before* ranking. If it happened after, a chunk from another village would compete for the top-k slots and the filter would remove it, returning fewer results. Worse, any bug in the post-filter would leak data. Filtering the candidate set first means a similar enough vector can never become a route around permissions.

Graph expansion

For each top hit, up to three linked neighbours are pulled in, but only if they were already in the visible set. The neighbour is labelled with *why* it is present ("included because it is in village ..."), which stops the model treating incidental context as the subject of the question. This is a one-hop breadth-first expansion with a fan-out cap.

The privacy boundary: the model may see the village, never the villager

Google's free-tier terms allow submitted content to be used to improve their products. A resident applying for a widow's pension cannot meaningfully consent to that. Three mechanisms enforce the boundary in code:

- **Residents are never indexed.** Indexing runs over every row whether or not anyone asks a question, so indexing residents would export the register as a standing cost of the feature.
- **Personal facts are flagged at retrieval.** `Retrieved.has_personal` is sticky: a set of facts is only as shareable as its most sensitive member. The assistant refuses to build a prompt from such a set and answers from the records directly instead.
- **The decision was never the model's.** Eligibility comes from the rule engine, so withholding the record costs phrasing and nothing else.

Anonymisation was considered and rejected. At village scale "a 52-year-old female agricultural labourer in ward 3" identifies one person in a few hundred; stripping the name leaves the record re-identifiable from its quasi-identifiers. This is the k-anonymity problem, and it is a strong point to raise in an interview.

Graceful degradation

If there is no index, no API key, an embedding error or no hit above the 0.55 floor, the assistant falls back to keyword routing over the same scoped queries. If generation fails, `plain_answer()` renders the retrieved facts directly. The UI shows which path ran, so nobody has to guess. A bug found only by running the app: for a citizen, the semantic path returned scheme descriptions instead of their own eligibility, because it returned before the rule engine ran. The fix layers semantic hits on top of the keyword pass for citizens.

Resilient model calls

`llm.generate()` retries 429, 500, 502, 503 and 504 twice with delays of 0.6 s and 1.8 s, and treats transport errors the same way. A 404 is deliberately not retried, because a retired model name never recovers. Errors include Google's own message plus a hint, since a bare "status 404" sends people checking their key when the real problem is the model name. A separate `/assistant/model-check` endpoint makes one real call, because `/health` only knows whether a key is set.

9. Grievance classification

A transparent, rule-based classifier over bilingual (English and Marathi) keyword sets. It is **not** a trained model, and the project says so plainly. It runs server-side so the same rules apply whether a complaint arrives from the citizen portal, an officer's desk or the API.

- **Category:** the keyword set with the most hits wins (Water, Sanitation, Roads, Electricity, Health), ties go to the first listed, no hits means Other.
- **Priority:** escalation terms first (contamination, collapse, electrocution, outbreak, injury, fire gives Critical; urgent, overflow, burst, school gives High), then category defaults. Drinking water and health default up, not down.
- **Routing:** each category maps to a department, in both languages.
- **Explainability:** the matched terms are returned, so an officer can see why a complaint was routed where it was.

Why rules rather than ML? There is no labelled dataset of Haveli grievances to train on, five categories are a small closed set, and a misrouted complaint needs a visible reason. The retrieval layer imports the same keyword list, so a word added for classification immediately helps the assistant find matching complaints, and the two cannot drift apart.

Complexity: naive substring search is $O(K \times L)$ for K keywords and text length L . With roughly 100 keywords and short complaints this is negligible. Aho-Corasick would make it $O(L + \text{matches})$ and is discussed in Section 15.

10. Document readers and resident sign-up

Gram Sabha transcript reader

Text is extracted from the actual uploaded file (PDF via PyMuPDF, DOCX via python-docx, or plain text), capped at 30,000 characters, and sent to Gemini with a **response schema**: title, summary, decisions and action items, each in English and Marathi. Constraining the output to a JSON schema removes a whole class of parsing failures. The system prompt forbids inventing names, dates or amounts not in the text. The earlier stub returned the same summary whatever was uploaded.

Government Resolution to scheme

An officer uploads a GR (a published state order announcing a scheme). The model proposes a scheme record with machine-readable criteria. Three safeguards make this safe enough to use:

- **Human approval gate.** The proposal is saved as *pending* and reaches no resident until an officer approves it. A model that read "60 years" as a maximum instead of a minimum would invert the scheme for everyone.
- **Closed vocabulary.** Any criteria key the engine does not understand is stripped and reported, not stored. An unknown key would be silently ignored at evaluation time, making the scheme look stricter on paper than it behaves.
- **Confidence capped at medium.** Nothing read by a model deserves the standing of a hand-verified scheme.

Resident sign-up and record matching

A resident applies; an officer matches them to the village register from a ranked candidate list and approves. Applying never creates a working login, and officer accounts are created only by an admin. The ranking is a weighted score:

Signal	Weight	Comment
Applicant quoted this citizen ID	+1.0	Strongest signal, still only a suggestion
Last 10 phone digits match	+0.8	Digits only, so formatting and +91 prefixes do not matter
Name similarity ≥ 0.6	+0.6 x ratio	difflib.SequenceMatcher (Ratcliff/Obershelp)
Same ward	+0.15	Weak tie-breaker
Threshold	≥ 0.4	Top 8 shown, confidence capped at 1.0, existing accounts flagged

The function only ranks; a human decides. An unconvincing list is the correct output when the applicant is not in the register.

11. The audit trail

`audit_events` answers "who opened Savita's file, and when?". The key design decision is that it is written by **middleware**, not by calls scattered through the routes. A trail built from per-route calls is only as complete as the last developer remembered to make it, and the endpoint that gets forgotten is always the new one. Here a request is recorded because it was served.

Recorded	Not recorded, deliberately
Every state change by a signed-in user	Unauthenticated requests: there is nobody to attribute them to
Every read that names one record	List endpoints: they would bury the events worth finding
Refused attempts, with their status code	Request bodies, ever: no incomes, contents or passwords in a log

- **Entity detection by ordered regex.** Patterns are checked longest-first, so `/citizens/{id}/documents` is filed under the citizen rather than swallowed by a shorter pattern.
- **Default is to record.** Read-only POSTs (the assistant's `/ask` and `/context`) must be named explicitly to be exempt, so forgetting the list makes the trail noisier, never emptier.
- **It can never break a request.** Failures to write are logged and swallowed. Losing one audit row is bad; refusing a grievance because the audit table is full is worse.
- **Admin-only and append-only.** No endpoint edits or deletes an event. `prune()` drops rows older than a year.

12. The frontend

Twenty-two components cover the officer workspace (dashboard, citizens, grievances, projects, Gram Sabha, GIS, analytics, assistant, registration review, audit trail, district overview) and the citizen portal (own file, schemes, grievances).

A typed client with one transparent refresh

`src/lib/api.ts` wraps `fetch`. Any non-2xx response throws an `ApiError` carrying the server's own message, so a failed save looks failed. On a 401 the client tries the refresh token once and replays the original request; if that also fails, the session-expired handler signs the user out.

Two hooks instead of a state library

`src/lib/useApi.ts` (abridged)

```
export function useQuery<T>(fetcher: () => Promise<T>, deps: unknown[] = []) {
  const [data, setData] = useState<T | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const [tick, setTick] = useState(0);
  const fetcherRef = useRef(fetcher); fetcherRef.current = fetcher;

  useEffect(() => {
    let cancelled = false; // ignore stale responses
    setLoading(true); setError(null);
    fetcherRef.current()
      .then(r => { if (!cancelled) setData(r); })
      .catch(e => { if (!cancelled) setError(errorMessage(e)); })
      .finally(() => { if (!cancelled) setLoading(false); });
    return () => { cancelled = true; };
  }, [...deps, tick]);

  return { data, loading, error, refetch: useCallback(() => setTick(n => n + 1), []) };
}
```

Two details an interviewer may probe: the **cancelled flag** prevents a slow earlier response from overwriting a newer one (a race condition when dependencies change quickly), and the **fetcher ref** lets callers pass inline arrow functions without triggering an infinite effect loop.

Maps, charts and language

- **GIS:** Leaflet over OpenStreetMap tiles; markers come from the API's own coordinates. Villages without a verified centre make the map fit to whatever is plotted, and the screen says so. Wadhu Khurd is left unplaced on purpose because two villages with near-identical names sit a few hundred metres apart: an unplaced village is honest, a misplaced one is not.
- **Charts:** Recharts for analytics dashboards.
- **Language:** i18next with English and Marathi resource bundles; the backend returns paired fields (name / nameMr).

13. Testing, deployment and operations

Testing

The backend suite has **284 tests, all passing** (about 53 seconds locally). Tests run against SQLite for speed, which is why `as_utc()` and the `none_as_null` fix exist: dialect differences were caught by tests rather than by users.

File	Focus
<code>test_api.py</code>	Every resource end to end: create, read, update, role refusals, eligibility verdicts
<code>test_semantic_retrieval.py</code>	Cosine maths, scoping before ranking, expansion limits, fallbacks, no residents in the index
<code>test_password_reset.py</code>	Permissions, expiry, single use, alphabet, session revocation
<code>test_village_isolation.py</code>	The neighbouring officer, route by route
<code>test_registration.py</code>	Applications, candidate ranking, approval, no login before approval
<code>test_audit.py</code>	What is and is not recorded, entity detection, failure isolation
<code>test_scheme_reader.py</code>	Closed vocabulary stripping, pending status, confidence cap
<code>test_rate_limit.py</code>	Thresholds, lockout expiry, unknown-email parity
<code>test_llm_retry.py</code>	Retry on 503, no retry on 404, transport errors
<code>test_seed_data.py</code>	Seed integrity: real LGD codes, coordinates inside the Haveli bounding box

Deployment

- **render.yaml** declares two services: the API (Docker, Singapore region, health check on `/health`) and the static site.
- **Migrations on boot:** the container runs `alembic upgrade head` before `uvicorn` starts with two workers.
- **Secrets** are marked `sync: false`, so they live in Render, never in the public repository.
- **VITE_API_URL** is baked in at build time, so changing it needs a rebuild, not a restart.
- **Fingerprinted assets** are cached for a year with `immutable`.
- **Keep-alive:** the README specifies a GitHub Action (`.github/workflows/keepalive.yml`) that pings the API every six hours so Supabase does not pause the free project. Note that this workflow file is not yet committed to the repository, so it needs adding before relying on it.

The CORS trap. If `CORS_ORIGINS` is wrong, the site loads, every request fails, the browser shows a CORS error and the API log shows nothing, because the browser blocked the request before it was sent. Knowing why the server log is empty is a good sign of real debugging experience.

14. Honest limitations and how it would scale

What the system does not have

- No trained or fine-tuned model of its own; the classifier is rules, the engine is rules.
- No OCR for scanned documents, no Aadhaar or DigiLocker integration, no SMS or payment gateway.
- Two languages only. A resident must visit the office to reset a password.
- The free API instance sleeps after 15 minutes; the first request after that waits roughly a minute.
- Grievance free text and uploaded minutes can still carry names to the model; structured fields are controlled, prose is not.

From one block to a whole state

Pressure point	Today	At district or state scale
Vector search	Linear cosine scan in Python over about 67 chunks	pgvector column with an HNSW or IVFFlat index behind the same semantic_search() function
Eligibility screening	On request, $O(C \times S \times R)$	Precompute verdicts in a background job, invalidate on record or scheme change
Rate limiting	COUNT over auth_attempts	Redis with a sliding-window or token-bucket counter; keep the table for audit
Audit trail	Synchronous insert in middleware	Write to a queue (Pub/Sub, Kafka) and batch into a partitioned table
Sessions	Stateless JWT + tokens_valid_from	Same design; add key rotation with a kid header
Model calls	One free key, retries	Paid tier with data-use guarantees, request queue, per-user quotas, response caching
Cold starts	Free instance sleeps	Always-on instances behind a load balancer; database read replicas

15. Interview question bank: data structures and algorithms

These are the questions a senior engineer is most likely to build out of this specific project. Each answer is written the way you would say it out loud: the direct answer first, then the reasoning, then the trade-off.

Q1. What is the time complexity of your semantic search, and how would you improve it?

Today it is $O(N \times d)$ to score N chunks of dimension d , then $O(N \log N)$ to sort. With N about 67 and $d = 768$ that is around fifty thousand multiply-adds, well under a millisecond.

Two improvements. First, we only need the top k , so a min-heap of size k gives $O(N \log k)$ instead of a full sort. Second, if vectors are normalised once at index time, cosine becomes a plain dot product, removing two square roots per comparison. Beyond about 100,000 chunks I would move to approximate nearest neighbour search (HNSW or IVFFlat in pgvector), which is roughly logarithmic per query at the cost of occasionally missing a true neighbour.

```
import heapq
def top_k(query, chunks, k=6, floor=0.55):
    heap = []                                # min-heap of (score, idx)
    for i, c in enumerate(chunks):
        s = dot(query, c.unit_vec)           # pre-normalised -> cosine
        if s < floor:
            continue
        if len(heap) < k:
            heapq.heappush(heap, (s, i))
        elif s > heap[0][0]:
            heapq.heapreplace(heap, (s, i))
    return sorted(heap, reverse=True)        #  $O(N \log k)$  overall
```

Q2. Why cosine similarity and not Euclidean distance?

Cosine measures direction and ignores magnitude, and for text embeddings the direction carries the meaning. For unit vectors the two are equivalent for ranking, because $\|a - b\|^2 = 2 - 2\cos(a, b)$. So normalising and then using either gives the same order.

Q3. Explain HNSW versus IVFFlat as if I were choosing for this project.

IVFFlat clusters vectors with k -means into lists and searches only the nearest few lists. It builds quickly and uses little memory, but recall depends on how many lists you probe and it needs data present before the index is built.

HNSW builds a layered proximity graph and searches it greedily from the top layer down. It has better recall at the same speed and handles inserts well, at the cost of more memory and slower builds. For a growing index with frequent small updates, like grievances being filed daily, I would choose HNSW.

Q4. Your graph expansion: which traversal is it, and what is its cost?

It is a one-hop breadth-first expansion with a fan-out cap of three links per hit and a visited set. With k hits the cost is $O(k \times 3)$ dictionary lookups after an $O(N)$ pass to build the (type, id) map. The visited set prevents duplicates; the visibility map prevents expansion becoming a route around permissions. Going to two hops would be a standard BFS with a depth limit, but the context would grow quickly and dilute relevance, so the cap is a deliberate precision choice.

Q5. The grievance classifier does substring search for about 100 keywords. How would you make it scale to 10,000 keywords?

Build an Aho-Corasick automaton: a trie of all keywords with failure links. Matching then takes $O(L + z)$ for text length L and z matches, independent of the number of keywords, after $O(\text{total keyword length})$ preprocessing. Today's naive approach is $O(K \times L)$.

```
from collections import deque
def build(words):
    goto, fail, out = [{}, [0], [set()]]
    for w in words:
        s = 0
        for ch in w:
            if ch not in goto[s]:
                goto.append({}); fail.append(0); out.append(set())
                goto[s][ch] = len(goto) - 1
            s = goto[s][ch]
        out[s].add(w)
    q = deque(goto[0].values())
    while q:
        r = q.popleft()
        for ch, s in goto[r].items():
            q.append(s)
            f = fail[r]
            while f and ch not in goto[f]:
                f = fail[f]
            fail[s] = goto[f].get(ch, 0) if goto[f].get(ch, 0) != s else 0
            out[s] |= out[fail[s]]
    return goto, fail, out

def search(text, goto, fail, out):
    s, found = 0, set()
    for ch in text:
        while s and ch not in goto[s]:
            s = fail[s]
        s = goto[s].get(ch, 0)
        found |= out[s]
    return found
```

Q6. Your rate limiter counts rows in a fixed lookback window. Compare that with a token bucket and a sliding window counter.

What we have is effectively a **sliding log**: every attempt is stored with a timestamp and we count those inside the last 15 minutes. It is exact and doubles as an audit trail, but each check is a COUNT query, $O(\log n)$ with an index on (email, created_at).

A **fixed window** counter is $O(1)$ but allows a burst of 2x the limit across a window boundary. A **sliding window counter** approximates the log by weighting the previous window's count, $O(1)$ memory per key. A **token bucket** refills at a steady rate and allows controlled bursts, $O(1)$ per key. At scale I would put a token bucket or sliding counter in Redis and keep the table for audit only.

```
import time
class TokenBucket:
    def __init__(self, capacity, refill_per_sec):
        self.cap, self.rate = capacity, refill_per_sec
        self.tokens, self.last = capacity, time.monotonic()
    def allow(self):
        now = time.monotonic()
        self.tokens = min(self.cap, self.tokens + (now - self.last) * self.rate)
        self.last = now
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False
```

Q7. Is there a race condition in the throttle? Two wrong guesses arrive at the same instant.

Yes, a small one. Both requests can read a count of four, both pass the check, both run bcrypt, and both record a failure, so the attacker gets six guesses instead of five. The window of opportunity is one bcrypt verification long. It is acceptable here because the goal is to make guessing impractical, not to enforce an exact number. If exactness mattered, I would use an atomic increment (Redis INCR, or an UPSERT with RETURNING on a counter row) and check the returned value.

Q8. How many reset codes are possible, and is that enough?

30 characters over 10 positions gives 30^{10} , about 5.9×10^{14} , roughly 49 bits. Redemption is limited to a handful of attempts per window, and the code expires in 24 hours. Even ignoring expiry, at five guesses per 15 minutes an attacker would need on the order of 10^{12} years for a 50 percent chance. The entropy is far larger than the attempt budget, which is the right relationship.

Q9. You use `difflib.SequenceMatcher` for name matching. What does it compute, and what would you use instead?

It is the Ratcliff/Obershelp algorithm: it recursively finds the longest common substring and scores $2M / T$, where M is matched characters and T is the total length. Worst case is roughly quadratic, which is fine for short names.

Alternatives: Levenshtein edit distance (dynamic programming, $O(m \times n)$), Jaro-Winkler, which favours common prefixes and suits personal names, or a phonetic encoding for Indian names transliterated in different ways (Patil, Paatil). For many candidates I would first block on ward or phone digits to avoid comparing against everyone.

```
def levenshtein(a, b):
    prev = list(range(len(b) + 1))
    for i, ca in enumerate(a, 1):
        cur = [i]
        for j, cb in enumerate(b, 1):
            cur.append(min(prev[j] + 1,          # deletion
                           cur[j - 1] + 1,      # insertion
                           prev[j - 1] + (ca != cb)) # substitution)
        prev = cur
    return prev[-1]          # O(m*n) time, O(n) space
```

Q10. The `any_of` evaluation is recursive. Could that blow the stack?

In principle a maliciously nested criteria dictionary could exceed Python's default recursion limit of about 1,000. In practice criteria are written by officers or by the scheme reader, which strips unknown keys. The fix is a depth limit at validation time, or converting the evaluation to an explicit stack. Real schemes nest at most two levels.

Q11. Why do you hash the entity key with SHA-1 to make chunk IDs? Isn't SHA-1 broken?

SHA-1 is broken for collision resistance against an adversary, but here it is not a security primitive. It is a deterministic way to turn 'grievance:griev_201' into a fixed-length ID, so re-indexing updates the same row instead of creating a duplicate. The inputs are our own keys, not attacker-controlled, and the unique constraint on (entity_type, entity_id) is the real guarantee. Any stable hash would do.

Q12. How does the audit middleware decide which record a path refers to, and what is the complexity?

It tries an ordered list of compiled regular expressions, longest and most specific first, and takes the first match. With P patterns and a path of length L that is $O(P \times L)$ in the worst case, which is tiny. Order matters because the shorter `/citizens/{id}` pattern would otherwise capture `/citizens/{id}/documents`. A trie of path segments would be the scalable alternative, and is essentially what routers use.

Q13. Document matching uses bidirectional substring checks after normalisation. What can go wrong?

Very short names can over-match: a document typed as 'ID' is a substring of many requirement names. Mitigations are a minimum length before substring matching, token-level matching instead of character-level, or a canonical document-type table so uploads choose from a list rather than free text. The last one is the right long-term fix.

Q14. What data structures would you use to compute the most complained-about ward per category in real time?

A hash map from (category, ward) to count gives $O(1)$ updates. For the maximum per category, keep a per-category max-heap with lazy deletion, or simply recompute the max over the small number of wards, since a village has under twenty. At district scale, a SQL GROUP BY with an index on (village_id, category, ward), or a materialised view refreshed on write, is simpler than a custom structure.

16. Interview question bank: system design and security

S1. Design E-Panchayat for all 27,000 Gram Panchayats in Maharashtra.

Requirements first: roughly 27,000 villages, tens of millions of residents, peak load on scheme deadlines, strict privacy, unreliable rural connectivity, and Marathi as the primary language.

Data: PostgreSQL partitioned or sharded by district, because almost every query is already scoped to one village and therefore one district. Village scoping becomes the shard key for free. Read replicas for analytics.

Services: keep the API stateless behind a load balancer and autoscale it. Move the eligibility screening, indexing and audit writes to asynchronous workers fed by a queue.

Retrieval: pgvector with HNSW per district partition, or a managed vector store, with the same scope-before-rank rule enforced as a filter in the index query.

AI: a paid, contractually private model tier or a self-hosted open model inside a government cloud, plus a prompt and response cache for repeated village-level questions.

Offline: a progressive web app with a service worker queue, so an officer can register a grievance without signal and sync later; conflicts resolved by last-writer-wins on independent fields and an explicit status-history log for grievances.

S2. Why is scoping before ranking safer than filtering after?

Post-filtering is a two-step contract where the second step is easy to forget, reorder or break. Pre-filtering makes the forbidden records simply absent from the computation. It also gives better results, because forbidden records cannot occupy top-k slots that are then thrown away. The principle is the same as parameterised SQL: make the unsafe state unrepresentable rather than detected.

S3. JWTs are stateless. How do you log someone out everywhere?

With a per-user revocation timestamp. Every request already loads the user to check `is_active`, so comparing the token's `iat` with `tokens_valid_from` costs nothing extra. It revokes all tokens for a user at once, which is exactly what a password reset needs. Revoking a single device would need a token ID (`jti`) denylist, typically in Redis with a TTL equal to the token's remaining lifetime.

S4. Walk me through what happens when an officer of Theur requests `GET /citizens/cit_102`, a Loni Kalbhor resident.

The request passes CORS, then the audit middleware wraps it. FastAPI resolves dependencies: the bearer token is decoded, its type checked, the user loaded, active status and revocation checked. The route calls `assert_can_read_citizen`, which computes the officer's scope (Theur), loads `cit_102`, sees village Loni Kalbhor, and raises 403. The middleware records a refused read against citizen `cit_102` with status 403. The response body says the record belongs to another Gram Panchayat. There is a test for exactly this case.

S5. What are the main OWASP risks for this system and where are they handled?

Broken access control (IDOR) by by-ID village guards; cryptographic failures by bcrypt and server-only secrets; injection by SQLAlchemy parameterised queries; identification and authentication failures by throttling, revocation and identical responses for unknown emails; security misconfiguration by exact CORS origins and secrets outside the repository; logging and monitoring by the audit trail. One honest gap: tokens are kept in localStorage, which is readable by any injected script, so a strict Content Security Policy would be the next step, or moving the refresh token to an HttpOnly cookie.

S6. Why not let the LLM decide eligibility? It reads the rules better than regex.

Because the output has to be reproducible, auditable and explainable to the person refused. The same inputs must give the same answer tomorrow and the officer must point to the rule. LLMs are non-deterministic and can hallucinate a threshold. We use the model where it is strong (reading a Government Resolution into a draft), and keep a human and a deterministic engine where the decision is made.

S7. How do you know the assistant is not hallucinating?

Four layers. The system prompt forbids facts outside the supplied records. Retrieval returns sources with every answer, so a claim can be checked against its row. A similarity floor of 0.55 keeps weak matches out, because the model treats anything it is given as relevant. And anything about an individual resident is answered from the records without the model at all. What we do not have, and would add, is an automated evaluation set of questions with expected source records, measuring retrieval recall and answer faithfulness.

S8. The free tier returns 503 under load. How did you handle it, and what would you do in production?

Retries with backoff on 429 and 5xx, no retry on 404, and graceful fallback to a records-only answer that the UI labels. In production: exponential backoff with jitter to avoid synchronised retries, a circuit breaker so a dead upstream fails fast, a per-user quota, and a paid tier. We also learned to pin a model version, because 'latest' aliases resolve to the newest and busiest model.

S9. Why is the audit trail middleware and not a decorator?

A decorator still has to be remembered on every new route; middleware does not. The failure mode of a forgotten decorator is a silent gap in the trail, exactly where nobody looks. The middleware default is to record, and exemptions must be named, so the failure mode of forgetting becomes extra noise rather than missing evidence.

S10. How would you add a third language, Hindi, without breaking anything?

Frontend: add a resource bundle to i18next. Backend: the paired name / name_mr columns do not scale, so I would move translatable text into a translations table keyed by (entity, field, locale) with fallback to English, migrate the Marathi columns into it with Alembic, and keep the API shape stable by resolving the requested locale server-side.

17. Coding exercises drawn from the project

Interviewers like to take a real component and ask you to write a clean version on the whiteboard. These five are the most likely, each with a solution and the follow-up you should expect.

Exercise 1: evaluate nested eligibility criteria

Given a resident as a dictionary and criteria with `min_age`, `max_income`, `requires_bpl` and nested `any_of` groups, return PASS, FAIL or UNKNOWN.

Three-valued AND across keys, three-valued OR inside any_of

```
PASS, FAIL, UNKNOWN = "PASS", "FAIL", "UNKNOWN"

def evaluate(person: dict, crit: dict) -> str:
    verdict = PASS
    def worse(a, b):
        # FAIL beats UNKNOWN beats PASS
        order = {PASS: 0, UNKNOWN: 1, FAIL: 2}
        return a if order[a] >= order[b] else b

    if "min_age" in crit:
        verdict = worse(verdict, PASS if person["age"] >= crit["min_age"] else FAIL)
    if "max_income" in crit:
        verdict = worse(verdict, PASS if person["income"] <= crit["max_income"] else FAIL)
    if crit.get("requires_bpl"):
        bpl = person.get("is_bpl")
        verdict = worse(verdict, UNKNOWN if bpl is None else (PASS if bpl else FAIL))

    if "any_of" in crit:
        results = [evaluate(person, b) for b in crit["any_of"]]
        group = PASS if PASS in results else (UNKNOWN if UNKNOWN in results else FAIL)
        verdict = worse(verdict, group)
    return verdict
```

Follow-up: "Make it iterative." Use an explicit stack of frames (criteria, partial results), or validate depth before evaluating. **Complexity:** $O(\text{number of rule nodes})$.

Exercise 2: a sliding-log rate limiter

Amortised $O(1)$ per call; $O(\text{limit})$ memory per key

```
from collections import defaultdict, deque
import time

class SlidingLogLimiter:
    def __init__(self, limit: int, window_s: float):
        self.limit, self.window = limit, window_s
        self.log = defaultdict(deque) # key -> timestamps of failures

    def allowed(self, key: str, now: float | None = None) -> bool:
        now = time.monotonic() if now is None else now
        q = self.log[key]
        while q and q[0] <= now - self.window: # evict expired entries
            q.popleft()
        return len(q) < self.limit

    def record_failure(self, key: str, now: float | None = None) -> None:
        self.log[key].append(time.monotonic() if now is None else now)
```

Follow-up: "Why record only failures?" Because counting refusals would renew the window forever. "How do you stop memory growing with many keys?" Periodically drop keys whose deque is empty, or use a TTL cache.

Exercise 3: single-flight token refresh

Five requests fail with 401 at once. Make sure only one refresh call is made and all five retry with the new token.

TypeScript: sharing an in-flight promise

```
let refreshing: Promise<boolean> | null = null;

async function refreshOnce(): Promise<boolean> {
  if (!refreshing) {
    refreshing = attemptRefresh().finally(() => { refreshing = null; });
  }
  return refreshing;          // every caller awaits the same promise
}

async function request<T>(path: string, init: RequestInit = {}): Promise<T> {
  let res = await send(path, init);
  if (res.status === 401 && await refreshOnce()) {
    res = await send(path, init);    // one retry with the new token
  }
  if (!res.ok) throw new ApiError(res.status, await messageFrom(res));
  return res.json() as Promise<T>;
}
```

This is an improvement over the current client, which retries once per request and could fire several refreshes in parallel. It is a good thing to volunteer: it shows you know where your own code could be better.

Exercise 4: top-k with graph expansion

$O(N \cdot d + N \log k + k \cdot \text{fanout})$

```
import heapq

def retrieve(q, chunks, links, k=6, fanout=3, floor=0.55):
    # chunks: id -> unit vector (already scoped); links: id -> [neighbour ids]
    scored = ((sum(a*b for a, b in zip(q, v)), cid) for cid, v in chunks.items())
    top = heapq.nlargest(k, (s for s in scored if s[0] >= floor))
    seen = {cid for _, cid in top}
    expanded = []
    for score, cid in top:
        for n in links.get(cid, [])[:fanout]:
            if n in chunks and n not in seen:    # must already be visible
                seen.add(n)
                expanded.append((score, n, f"via {cid}"))
    return top, expanded
```

Exercise 5: merge overlapping project timelines

Given development projects as (start, end) dates, return the periods when at least one project was active, to show on the analytics page.

Classic interval merge

```
def merge(intervals):  
    intervals.sort()                #  $O(n \log n)$   
    out = []  
    for s, e in intervals:  
        if out and s <= out[-1][1]:  
            out[-1][1] = max(out[-1][1], e)  
        else:  
            out.append([s, e])  
    return out
```

Variant: "Maximum number of projects running at once" is a sweep line: +1 at each start, -1 at each end, sort events, track the running maximum.

18. Behavioural and viva questions

Senior engineers ask these to see judgement, ownership and honesty. Good answers are specific and include what went wrong.

B1. Tell me about a bug you are proud of finding.

The village isolation gap. List endpoints were scoped, so the neighbouring officer saw an empty list and every test passed. But records could still be opened by ID, and our IDs are guessable. It went unnoticed because all ten seeded residents lived in one village, so the fixtures could not express the bug. We fixed it centrally in `deps.py`, so new routes cannot forget it, and wrote 24 tests that sign in as the officer next door and try every by-ID route.

B2. Tell me about a decision you would make differently.

Keeping tokens in `localStorage`. It was the quickest way to survive a page reload, but it exposes them to any injected script. With more time I would move the refresh token into an `HttpOnly`, `SameSite` cookie and keep the access token in memory only.

B3. How did you decide what not to build?

We wrote an honest capability statement in the README and checked every feature against it: no trained model, no OCR, no Aadhaar, no SMS. Saying 'AI classifier' about a keyword list would have been easy and wrong. Being exact about what the system is protects the people who would rely on it.

B4. What was the hardest trade-off?

Privacy against answer quality in the assistant. Sending a resident's record to the model would produce nicer sentences. We decided the model may see the village and never the villager, because the eligibility decision was never the model's anyway. We lose some fluency and gain a guarantee that does not depend on a vendor's terms.

B5. How did you work as a team?

Small, reviewable commits with messages that explain why, a README that doubles as the design document, and tests written alongside each fix so a regression would be caught by someone else's change. Infrastructure is in `render.yaml` so nothing depends on one person remembering dashboard settings.

Quick-fire viva answers

Question	One-line answer
What is RAG?	Retrieve relevant records first, then generate an answer constrained to them, with sources.
What is an embedding?	A fixed-length vector (768 floats here) whose direction captures the meaning of a text.
Why bcrypt?	Deliberately slow, salted hashing; each guess is expensive and identical passwords hash differently.
What is LGD?	The Local Government Directory: official codes for every Indian state, district, block and village.
Why FastAPI's Depends?	Composable guards; auth and scoping are declared per route and cannot be skipped by the handler.

Question	One-line answer
What is CORS?	A browser rule that blocks cross-origin requests unless the server lists the calling origin.
What does Alembic do?	Versioned, reversible database schema migrations, applied automatically on deployment.
Why HTTP 429?	Too Many Requests, with a Retry-After header telling the client when to try again.
What is IDOR?	Opening another user's record by changing an ID; prevented by checking ownership on every by-ID route.
What is SECC-2011?	The Socio-Economic Caste Census, used by PMAY-G and Ayushman Bharat to identify deprived households.
Why a response schema?	It forces the model to return valid JSON of a known shape, removing parsing failures.
Why a similarity floor?	Weak matches are worse than none; the model treats whatever it receives as relevant.

19. Glossary

Term	Meaning
Gram Panchayat	Elected village-level local government in India
Gram Sabha	Assembly of all adult residents of a village; its decisions are public record
GR (Shasan Nirnay)	Government Resolution: a published state order, often announcing or changing a scheme
BPL	Below Poverty Line list, a common eligibility gate
Ration card types	Yellow, Orange, AAY (Antyodaya), Annapurna, White; several health schemes depend on them
7/12 extract	Maharashtra's land record document, used to prove land holding
Haveli	A taluka (block) of Pune district; the project's 23 villages are here
Access / refresh token	Short-lived credential for API calls / longer-lived credential used only to get new access tokens
IDOR	Insecure Direct Object Reference: reaching a record you should not by guessing its ID
k-anonymity	A record is k-anonymous if at least k people share its quasi-identifiers; tiny villages make k small
HNSW / IVFFlat	Approximate nearest-neighbour index types available in pgvector
Aho-Corasick	Multi-pattern string matching in time linear in the text
Fail closed	When something is missing or broken, deny access rather than grant it

This handbook describes the repository at commit 897818c. If the code changes, the section on the module concerned should be updated in the same pull request, the same way the project keeps its README in step with its code.